Solution: Maximum Level Sum of a Binary Tree

Problem Statement

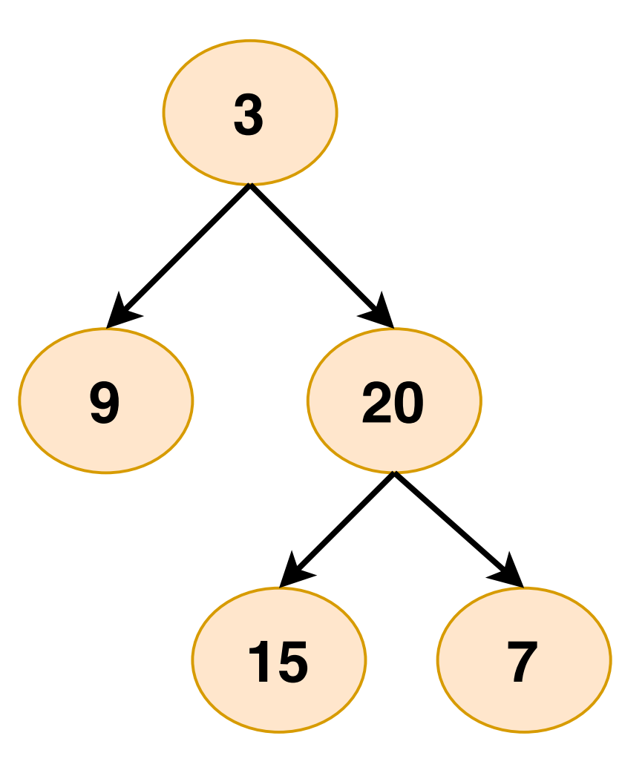
Given a `root` of the binary tree, return the s`mallest` level `x`, such that sum of all node's values at level `x` has maximum sum.

The level of a binary tree starts at 1 for the root node, and increases by 1 for each subsequent depth.

Examples

Example 1:

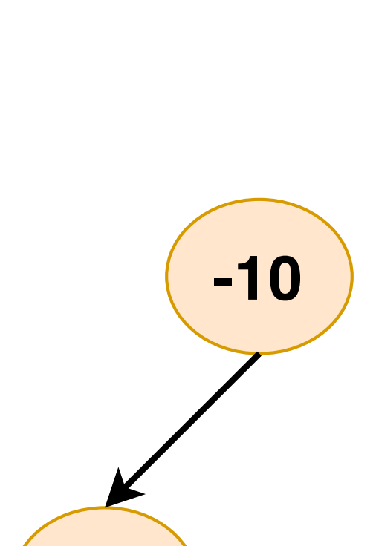
- **Input**: [3,9,20,null,null,15,7]



- **Expected Output**: 2
- **Justification**: The sum of each level is: Level 1: 3, Level 2: 29, Level 3: 22. The second level has the maximum sum, so the output is **2**.

Example 2:

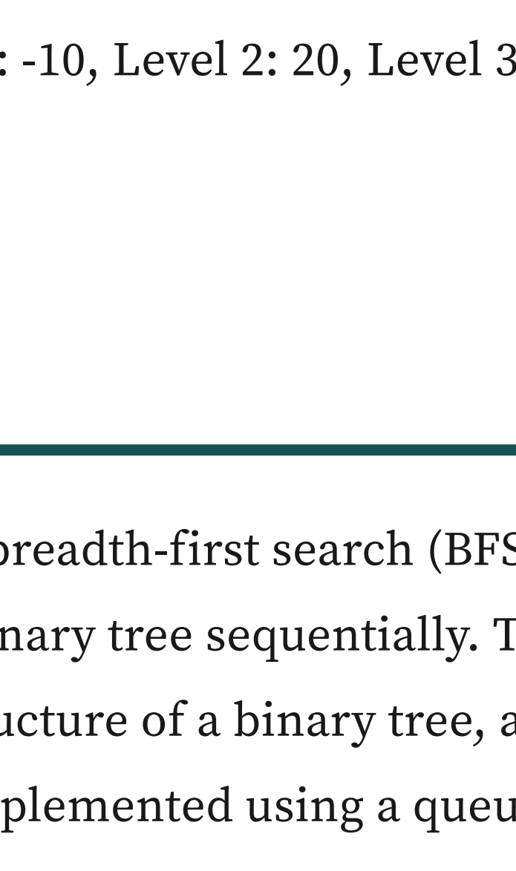
- **Input**: [1,2,3,4,5,null,7]



- **Expected Output**: 3
- **Justification**: The sums are Level 1: 1, Level 2: 5, Level 3: 16. The third level has the highest sum, thus the output is **3**.

Example 3:

- **Input**: [-10,20,null,15,25]



- **Expected Output**: 3
- **Justification**: The sums are Level 1: -10, Level 2: 20, Level 3: 40. The third level's sum is the highest, making the output **3**.

Solution

To solve this problem, we'll employ a breadth-first search (BFS) strategy, which is efficient for traversing through each level of the binary tree sequentially. This approach works well because it naturally follows the level-by-level structure of a binary tree, allowing us to calculate the sum of nodes at each level as we go. BFS is implemented using a queue that keeps track of nodes and their levels while traversing the tree. We believe this approach is the most effective because it directly aligns with the problem's requirement to evaluate each level's sum and identify the maximum among them without unnecessary computations or traversals.

Initially, we enqueue the root node along with its level. We then process each node in the queue, dequeue it, add its value to the current level's sum, and enqueue its children with the incremented level. This process is repeated until the queue is empty. Along the way, we keep track of the maximum sum encountered and its corresponding level. This method is efficient and straightforward, leveraging the inherent structure of the binary tree to address the problem directly.

Step-by-step Algorithm

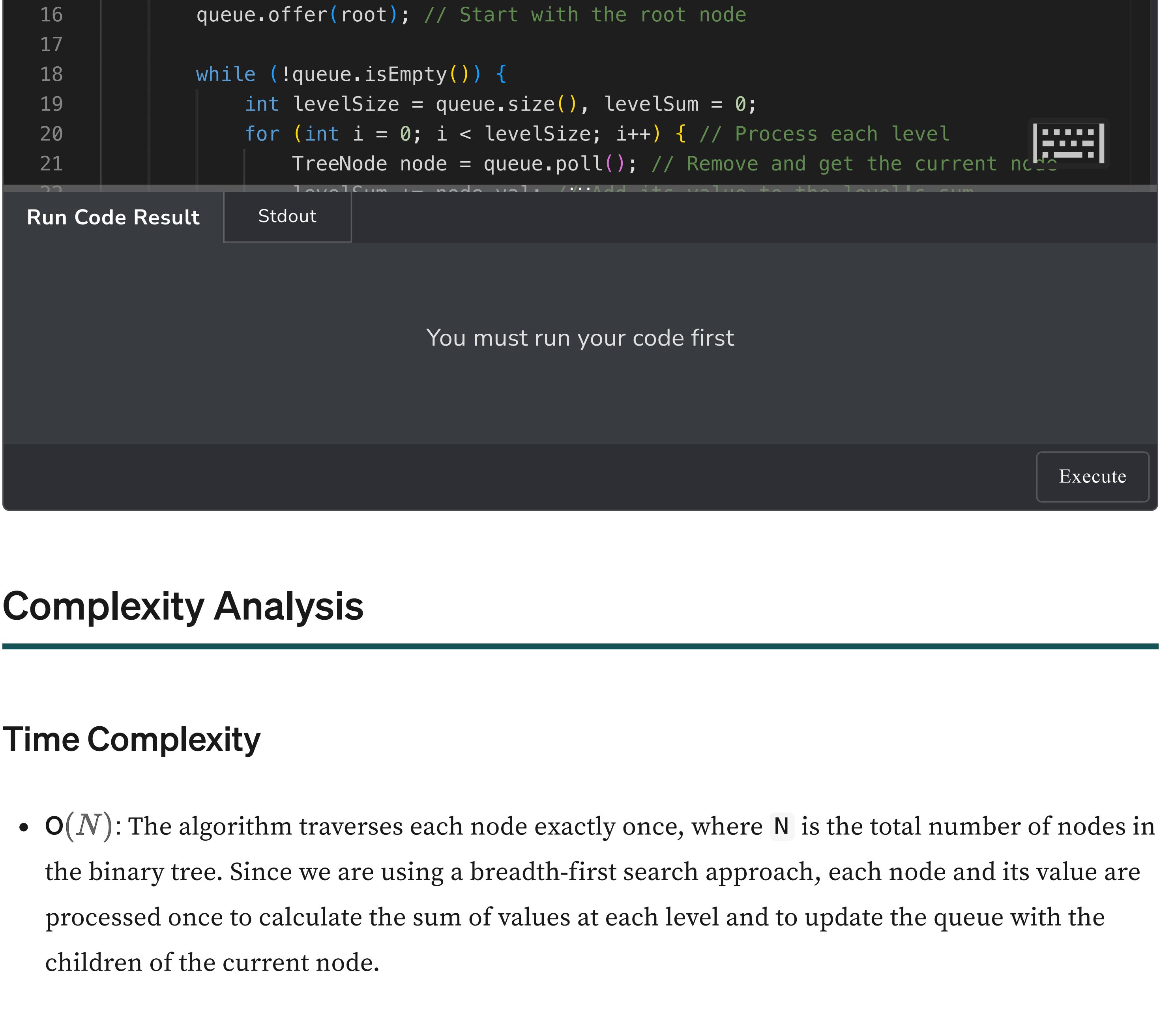
1. **Initialize Variables**:
    - Create a `Queue` to hold `TreeNode` objects, which will help us traverse the tree level by level. This queue will store nodes along with their corresponding level in the tree.
    - Define `maxSum` as `Integer.MIN_VALUE` to keep track of the maximum sum of node values found at any level.
    - Define `maxLevel` to keep track of the level with the maximum sum. Initialize it to 1, considering the root level as the first level.
    - Define `currentLevel` to keep track of the current level being processed. Initialize it to 1.
2. **Start with the Root Node**:
    - Add the root node to the queue if it is not null, to start the level order traversal.
3. **Level Order Traversal**:
    - While the queue is not empty, perform the following steps to process each level:
        - Determine the number of nodes at the current level (`levelSize`), which is the size of the queue at the start of the loop.
        - Initialize `levelSum` to 0 for accumulating the sum of node values at the current level.
        - Iterate through the nodes at the current level (i.e., loop from 0 to `levelSize` - 1):
            - Dequeue a node from the queue (`node`).
            - Add the node's value to `levelSum`.
            - If the node has a left child, enqueue the left child to the queue.
            - If the node has a right child, enqueue the right child to the queue.
        - After processing all nodes at the current level, compare `levelSum` with `maxSum`. If `levelSum` is greater, update `maxSum` with `levelSum` and set `maxLevel` to the current level (`currentLevel`).
        - Increment `currentLevel` to move to the next level in the tree.
4. **Return the Result**:
    - After completing the level order traversal, return `maxLevel`, which is the level with the maximum sum of node values.

Algorithm Walkthrough

Let's consider the input `[1,2,3,4,5,null,7]`.

1. **Initialization**:
    - `maxSum` = `Integer.MIN_VALUE` (the smallest value to start comparison).
    - `maxLevel` = 1 (assuming the root node is at level 1).
    - `currentLevel` = 1 (start from level 1).
    - Queue = [1] (root node is enqueued).
2. **Process Level 1**:
    - Start of Level 1:
        - `levelSize` = 1 (one node in the queue).
        - `levelSum` = 0 (initialize sum of levels).
    - Dequeue Node 1 (`val` = 1):
        - `levelSum` += 1.
        - Enqueue children of Node 1 (Nodes 2 and 3).
    - End of Level 1:
        - Check if `levelSum` > `maxSum`. Yes, so `maxSum` = 1, `maxLevel` = 1.
    - Increment `currentLevel` to 2.
3. **Process Level 2**:
    - Start of Level 2:
        - `levelSize` = 2 (two nodes in the queue, corresponding to nodes 2 and 3).
        - `levelSum` = 0.
    - Dequeue Node 2 (`val` = 2):
        - `levelSum` += 2.
        - Enqueue children of Node 2 (Nodes 4 and 5).
    - Dequeue Node 3 (`val` = 3):
        - `levelSum` += 3.
        - Enqueue child of Node 3 (Node 7; the left child is `null` and not enqueued).
    - End of Level 2:
        - `levelSum` = 5. Check if `levelSum` > `maxSum`. Yes, so `maxSum` = 5, `maxLevel` = 2.
    - Increment `currentLevel` to 3.
4. **Process Level 3**:
    - Start of Level 3:
        - `levelSize` = 3 (three nodes in the queue, corresponding to nodes 4, 5, and 7).
        - `levelSum` = 0.
    - Dequeue Node 4 (`val` = 4):
        - `levelSum` += 4.
    - Dequeue Node 5 (`val` = 5):
        - `levelSum` += 5.
    - Dequeue Node 7 (`val` = 7):
        - `levelSum` += 7.
    - End of Level 3:
        - `levelSum` = 16. Check if `levelSum` > `maxSum`. Yes, so `maxSum` = 16, `maxLevel` = 3.
    - There are no more nodes to process, so we're done.
5. **Final Result**:
    - The level with the maximum sum is Level 3 with a sum of 16.

Code



```java
import java.util.LinkedList;
import java.util.Queue;

// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; } // Constructor to create a new node
// }

public class Solution {
    public int maxLevelSum(TreeNode root) {
        if (root == null) return -1; // Return -1 if tree is empty
        int maxSum = Integer.MIN_VALUE, maxLevel = 1, currentLevel = 1;
        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root); // Start with the root node

        while (!queue.isEmpty()) {
            int levelSize = queue.size(), levelSum = 0;
            for (int i = 0; i < levelSize; i++) { // Process each level
                TreeNode node = queue.poll(); // Remove and get the current n
```

You must run your code first

Complexity Analysis

Time Complexity

- $O(N)$: The algorithm traverses each node exactly once, where `N` is the total number of nodes in the binary tree. Since we are using a breadth-first search approach, each node and its value are processed once to calculate the sum of values at each level and to update the queue with the children of the current node.

Space Complexity

- $O(N)$: In the worst case, the space complexity can be $O(N)$ when the tree is extremely unbalanced (e.g., all nodes are on one side of the tree).

← Previous
Maximum Level Sum Of A Binary Tree (Medium)

Next →
Multiply Strings (Medium)

☐ Mark as Completed